

Aplikasi CRUD HansInventory - CodeIgniter 4

Nama: Muhammad Daffa Bagus Jumanoro

NIM: 2311102222

Fakultas: Teknik Informatika - Telkom University

Daftar Isi

1. [Deskripsi](#)
 2. [Fitur Utama](#)
 3. [Teknologi](#)
 4. [Screenshots / Output Aplikasi](#)
 5. [Instalasi](#)
 6. [Login Default](#)
 7. [Struktur Database](#)
 8. [Struktur Folder](#)
 9. [Endpoint API](#)
 10. [Kode Program](#)
 11. [Cara Penggunaan](#)
 12. [Fitur Realtime](#)
 13. [Troubleshooting](#)
 14. [Referensi](#)
-

Deskripsi

Aplikasi web manajemen inventory dengan sistem autentikasi (login/register) dan fitur CRUD (Create, Read, Update, Delete) untuk mengelola data produk. Dibangun menggunakan framework CodeIgniter 4 dengan pendekatan RESTful API dan tema glassmorphism.

Fitur Utama

1. Sistem Autentikasi

- Registrasi pengguna baru
- Login dengan username atau email
- Logout dengan konfirmasi
- Role user (admin/user)

2. Manajemen Produk

- Menambah produk baru
- Menampilkan daftar produk dalam tabel interaktif (DataTables)
- Mengedit data produk
- Menghapus produk dengan konfirmasi SweetAlert2

3. Dashboard Statistik

- Total produk
- Total nilai inventory
- Alert stok rendah ($\text{stok} < 10$)
- Grafik distribusi stok (Chart.js)
- Update statistik realtime tanpa reload

4. Fitur Tambahan

- Export data ke Excel
- Pencarian data realtime (DataTables)
- Tema glassmorphism dengan gradient
- Responsive design (Bootstrap 5)
- Animasi fade-in pada halaman

Teknologi

Backend:

- CodeIgniter 4.4
- MySQL 5.7+
- PHP 7.4 atau lebih tinggi

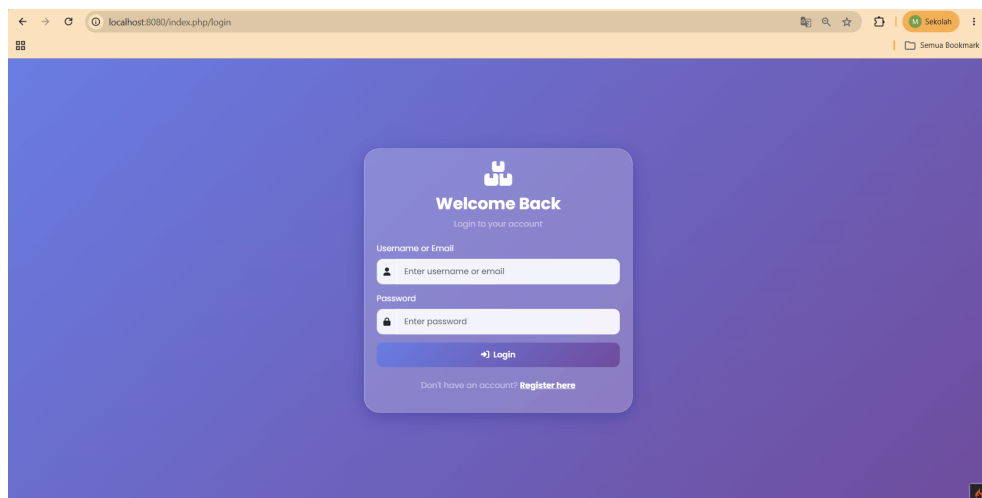
Frontend:

- Bootstrap 5.3

- jQuery 3.7
- DataTables 1.13
- Chart.js 4.4
- SweetAlert2 11
- Font Awesome 6
- Google Fonts (Poppins)

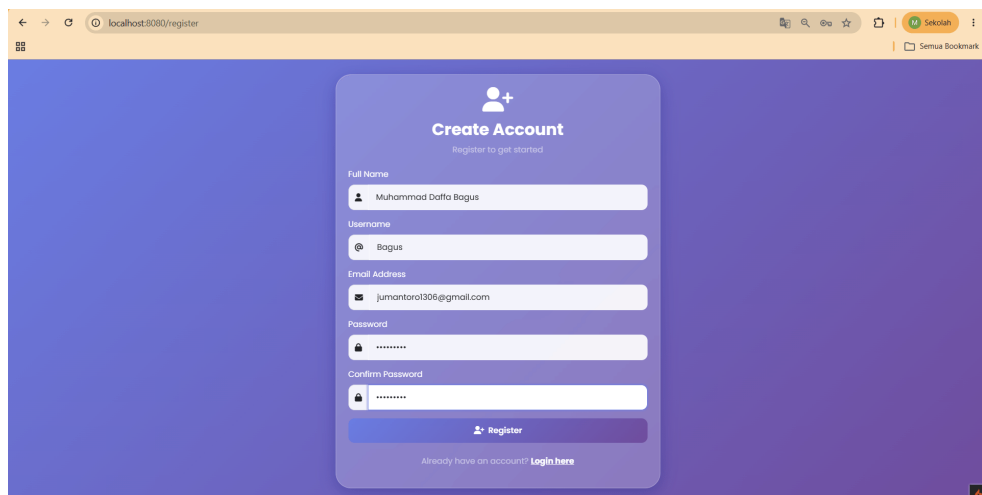
Screenshots / Output Aplikasi

1. Halaman Login



Halaman login dengan desain glassmorphism, gradient background ungu-biru, dan form input username/email serta password. Terdapat link untuk registrasi jika belum memiliki akun.

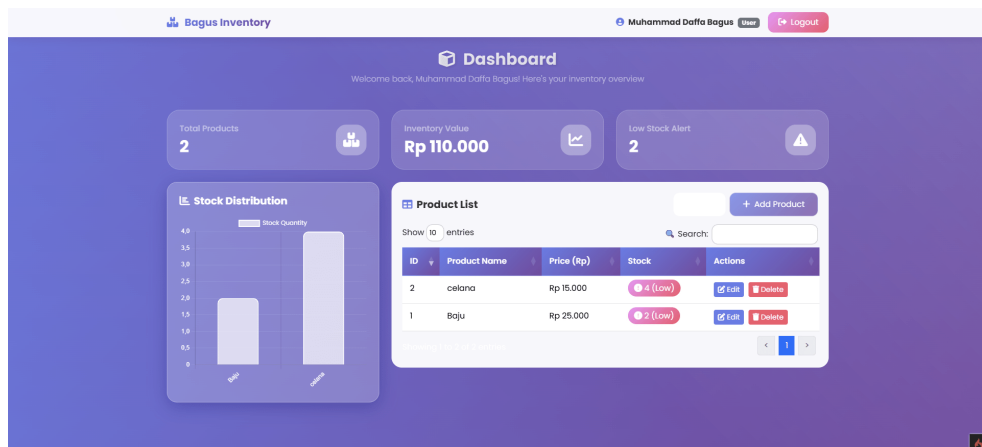
2. Halaman Register



Halaman registrasi pengguna baru dengan form input nama lengkap, username, email, password, dan konfirmasi password. Validasi dilakukan di

sisi server.

3. Dashboard



Halaman utama setelah login yang menampilkan:

- Informasi pengguna yang login
- Statistik total produk, total nilai inventory, dan alert stok rendah
- Grafik batang distribusi stok produk menggunakan Chart.js
- Tombol Add Product, Export Excel, dan Logout

4. Form Tambah Produk

The 'Product Form' modal is displayed over the dashboard, containing the following fields and controls:

- Product Name:**
- Price (Rp):**
- Stock:**
- Save Product:**

Modal form untuk menambah produk baru dengan input nama produk, harga, dan stok. Dilengkapi validasi input dan animasi smooth.

5. Pencarian Data dengan DataTables

Product List

+ Add Product

Show 10 entries

Search:

ID	Product Name	Price (Rp)	Stock	Actions
2	celana	Rp 15.000	4 (Low)	Edit Delete
1	Baju	Rp 25.000	2 (Low)	Edit Delete

Showing 1 to 2 of 2 entries

<

1

>

Tabel produk dengan fitur DataTables yang mendukung pencarian realtime, sorting, dan pagination. Data diambil secara async dari server.

6. Form Edit Produk

Product Form

×

Product Name

celana

Price (Rp)

15000,00

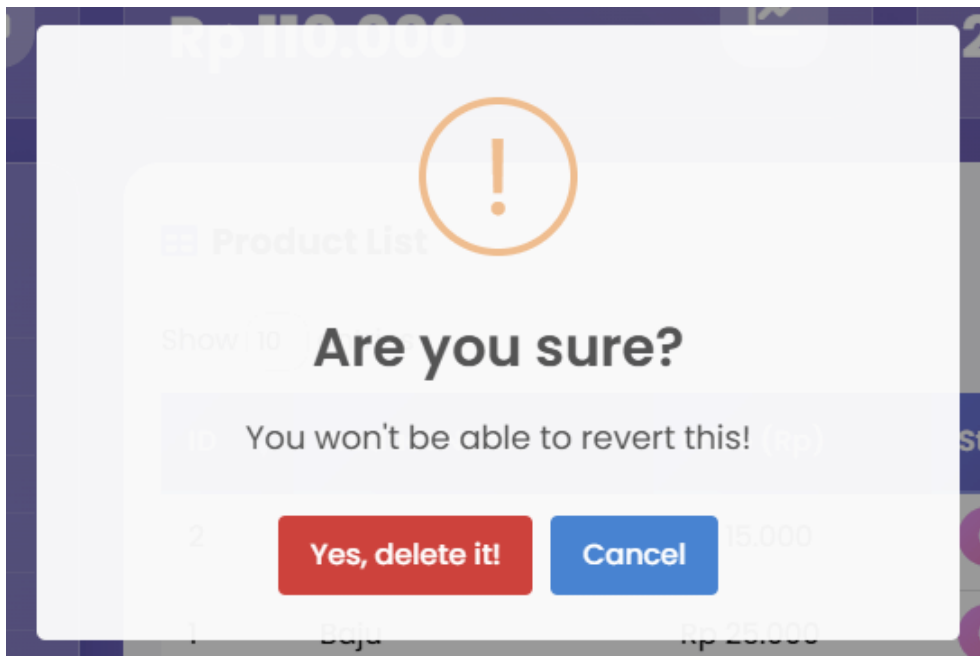
Stock

4

Save Product

Modal form untuk mengedit data produk yang sudah ada. Data terisi secara otomatis berdasarkan ID produk yang dipilih.

7. Hapus Produk dengan Konfirmasi



Konfirmasi penghapusan menggunakan SweetAlert2 dengan animasi dan tombol konfirmasi. Produk akan dihapus secara permanen setelah dikonfirmasi.

Instalasi

1. Clone repository

```
git clone https://github.com/YamaTaro38/tugas-cots:  
cd tugas-cots
```

2. Install dependencies

```
composer install
```

3. Konfigurasi database

```
cp env .env
```

Edit file `.env`:

```
CI_ENVIRONMENT = development  
database.default.hostname = localhost  
database.default.database = db_inventory  
database.default.username = root
```

```
database.default.password =  
database.default.DBDriver = MySQLi
```

4. Buat database

```
CREATE DATABASE db_inventory;  
USE db_inventory;
```

5. Import database (atau jalankan migration)

```
php spark migrate
```

6. Jalankan server

```
php spark serve
```

7. Akses aplikasi

Buka browser: <http://localhost:8080>

Login Default

Field	Value
Username	admin
Password	admin123

Atau registrasi akun baru melalui halaman register.

Struktur Database

Tabel users

Kolom	Tipe	Keterangan
id	int(11)	Primary key, auto increment
username	varchar(100)	Unique, untuk login

Kolom	Tipe	Keterangan
email	varchar(255)	Unique, untuk login
password	varchar(255)	Hash password
full_name	varchar(255)	Nama lengkap pengguna
role	enum	admin / user
is_active	tinyint(1)	1 = aktif, 0 = nonaktif
created_at	datetime	Waktu registrasi
updated_at	datetime	Waktu update terakhir

Tabel products

Kolom	Tipe	Keterangan
id	int(11)	Primary key, auto increment
product_name	varchar(255)	Nama produk
price	decimal(10,2)	Harga produk
stock	int(11)	Jumlah stok
created_by	int(11)	Foreign key ke users.id
created_at	datetime	Waktu input produk
updated_at	datetime	Waktu update terakhir

Struktur Folder

```
source code/                                # Root
direktori proyek
├─ output/                                  # Screenshot
hasil aplikasi
|   └─ 1.login.png
|   └─ 2.register.png
|   └─ 3.dashboard.png
|   └─ 4.form-add-product.png
|   └─ 5.search-datatable.png
|   └─ 6.form-edit-product.png
|   └─ 7.delete-product.png
|
```



```
├─ app/
│   ├── Config/
│   │   ├── Database.php
│   │   └── Routes.php
│   ├── Controllers/
│   │   ├── Auth.php
│   │   └── Product.php
│   ├── Models/
│   │   ├── UserModel.php
│   │   └── ProductModel.php
│   ├── Views/
│   │   ├── auth/
│   │   │   ├── login.php
│   │   │   └── register.php
│   │   ├── dashboard_view.php
│   │   ├── form_view.php
│   │   └── table_view.php
│   └── Database/
│       └── Migrations/
├─ public/
├─ writable/
├─ vendor/
├─ composer.json
└─ spark
```

Endpoint API

Method	Endpoint	Deskripsi
GET	/	Redirect ke login
GET	/login	Halaman login
POST	/login	Proses login
GET	/register	Halaman register
POST	/register	Proses registrasi
GET	/dashboard	Halaman dashboard
GET	/logout	Proses logout
GET	/get-data	Ambil semua data produk (JSON)
POST	/save	Simpan atau update produk

Method	Endpoint	Deskripsi
DELETE	/delete/{id}	Hapus produk berdasarkan ID
GET	/chart-data	Data untuk grafik (JSON)

Kode Program

Database SQL Lengkap

```
-- Drop database if exists
DROP DATABASE IF EXISTS db_inventory;
CREATE DATABASE db_inventory;
USE db_inventory;

-- Tabel users
CREATE TABLE `users` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `username` varchar(100) NOT NULL,
  `email` varchar(255) NOT NULL,
  `password` varchar(255) NOT NULL,
  `full_name` varchar(255) NOT NULL,
  `role` enum('admin','user') DEFAULT 'user',
  `is_active` tinyint(1) DEFAULT 1,
  `created_at` datetime DEFAULT NULL,
  `updated_at` datetime DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `username` (`username`),
  UNIQUE KEY `email` (`email`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Tabel products
CREATE TABLE `products` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `product_name` varchar(255) NOT NULL,
  `price` decimal(10,2) NOT NULL DEFAULT '0.00',
  `stock` int(11) NOT NULL DEFAULT '0',
  `created_by` int(11) DEFAULT NULL,
  `created_at` datetime DEFAULT NULL,
  `updated_at` datetime DEFAULT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
-- Insert admin default (password: admin123)
INSERT INTO `users` (`username`, `email`, `password`, `created_at`)
VALUES ('admin', 'admin@inventory.com', '$2y$10$92IXUNpkj80849LBDqMDB3gOvba2epGjKWxpXvVYgg9lcM', NOW());

-- Insert sample products
INSERT INTO `products` (`product_name`, `price`, `stock`, `created_at`)
VALUES ('MacBook Pro M3', 25000000, 12, NOW()),
('iPhone 15 Pro', 18000000, 8, NOW()),
('iPad Air', 9500000, 15, NOW()),
('AirPods Pro', 3200000, 25, NOW()),
('Apple Watch Series 9', 6500000, 6, NOW()),
('Samsung Galaxy S24', 12000000, 10, NOW()),
('Logitech MX Master 3S', 1500000, 20, NOW());
```

Controller Auth.php

```
<?php

namespace App\Controllers;

use App\Models\UserModel;

class Auth extends BaseController
{
    public function login()
    {
        if (session()->get('isLoggedIn')) {
            return redirect()->to('/dashboard');
        }
        return view('auth/login');
    }

    public function doLogin()
    {
        $model = new UserModel();
        $username = $this->request->getPost('username');
        $password = $this->request->getPost('password');

        if (empty($username) || empty($password)) {
            session()->setFlashdata('error', 'Username and password are required');
            return redirect()->back();
        }

        $user = $model->getUserByUsername($username);
        if (!$user) {
            session()->setFlashdata('error', 'User not found');
            return redirect()->back();
        }

        if (!password_verify($password, $user->password)) {
            session()->setFlashdata('error', 'Incorrect password');
            return redirect()->back();
        }

        session()->set('isLoggedIn', true);
        session()->set('user_id', $user->id);
        return redirect()->to('/dashboard');
    }
}
```

```

    }

    $user = $model->login($username, $password);

    if ($user) {
        session()->set([
            'user_id' => $user['id'],
            'username' => $user['username'],
            'email' => $user['email'],
            'full_name' => $user['full_name'],
            'role' => $user['role'],
            'isLoggedIn' => true
        ]);
        return redirect()->to('/dashboard');
    } else {
        session()->setFlashdata('error', 'User not found');
        return redirect()->back();
    }
}

public function register()
{
    if (session()->get('isLoggedIn')) {
        return redirect()->to('/dashboard');
    }
    return view('auth/register');
}

public function doRegister()
{
    $model = new UserModel();

    $rules = [
        'username' => 'required|min_length[3]',
        'email' => 'required|valid_email|is_unique',
        'password' => 'required|min_length[6]',
        'confirm_password' => 'required|matches',
        'full_name' => 'required|min_length[3]'
    ];

    if (!$this->validate($rules)) {
        return redirect()->back()->withInput($this->input());
    }
}

```

```

        $data = [
            'username' => $this->request->getPost('username'),
            'email' => $this->request->getPost('email'),
            'password' => password_hash($this->request->getPost('password'), PASSWORD_DEFAULT),
            'full_name' => $this->request->getPost('full_name'),
            'role' => 'user',
            'is_active' => 1
        ];

        if ($model->save($data)) {
            session()->setFlashdata('success', 'Registration successful');
            return redirect()->to('/login');
        } else {
            session()->setFlashdata('error', 'Registration failed');
            return redirect()->back();
        }
    }

    public function logout()
    {
        session()->destroy();
        return redirect()->to('/login');
    }
}

```

Controller Product.php

```

<?php

namespace App\Controllers;

use App\Models\ProductModel;

class Product extends BaseController
{
    protected $productModel;

    public function __construct()
    {
        $this->productModel = new ProductModel();
    }
}

```

```

public function dashboard()
{
    if (!session()->get('isLoggedIn')) {
        return redirect()->to('/login');
    }

    $products = $this->productModel->findAll();

    $totalProducts = count($products);
    $totalValue = 0;
    $lowStock = 0;

    foreach ($products as $p) {
        $totalValue += $p['price'] * $p['stock'];
        if ($p['stock'] < 10) {
            $lowStock++;
        }
    }

    $data = [
        'total_products' => $totalProducts,
        'total_value' => $totalValue,
        'low_stock' => $lowStock,
        'full_name' => session()->get('full_name'),
        'role' => session()->get('role')
    ];

    return view('dashboard_view', $data);
}

```

```

public function getData()
{
    $data = $this->productModel->findAll();
    return $this->response->setJSON($data);
}

```

```

public function save()
{
    $id = $this->request->getPost('id');
    $data = [
        'product_name' => $this->request->getPost('product_name'),
        'price' => $this->request->getPost('price'),
        'stock' => $this->request->getPost('stock'),
        'created_by' => session()->get('user_name')
    ];
}

```

```

];

if ($id && !empty($id)) {
    $this->productModel->update($id, $data);
    return $this->response->setJSON(['status' => 'success']);
} else {
    $this->productModel->insert($data);
    return $this->response->setJSON(['status' => 'success']);
}

}

public function delete($id)
{
    $this->productModel->delete($id);
    return $this->response->setJSON(['status' => 'success']);
}

public function getChartData()
{
    $products = $this->productModel->findAll();
    $labels = [];
    $stocks = [];

    foreach ($products as $p) {
        $labels[] = $p['product_name'];
        $stocks[] = (int)$p['stock'];
    }

    return $this->response->setJSON(['labels' => $labels, 'stocks' => $stocks]);
}
}

```

Model UserModel.php

```

<?php

namespace App\Models;

use CodeIgniter\Model;

class UserModel extends Model
{

```

```

        protected $table = 'users';
        protected $primaryKey = 'id';
        protected $allowedFields = ['username', 'email'];
        protected $useTimestamps = true;
        protected $createdField = 'created_at';
        protected $updatedField = 'updated_at';
        protected $returnType = 'array';

        public function login($username, $password)
        {
            $user = $this->where('username', $username)
                ->orWhere('email', $username)
                ->where('is_active', 1)
                ->first();

            if ($user && password_verify($password, $user->password)) {
                return $user;
            }

            return false;
        }
    }
}

```

Model ProductModel.php

```

<?php

namespace App\Models;

use CodeIgniter\Model;

class ProductModel extends Model
{
    protected $table = 'products';
    protected $primaryKey = 'id';
    protected $allowedFields = ['product_name', ''];
    protected $useTimestamps = true;
    protected $createdField = 'created_at';
    protected $updatedField = 'updated_at';
    protected $useSoftDeletes = false;
    protected $returnType = 'array';
}

```



```

        protected $validationRules = [
            'product_name' => 'required|min_length[3]',
            'price' => 'required|numeric|greater_than',
            'stock' => 'required|integer|greater_than',
        ];

        protected $validationMessages = [
            'product_name' => [
                'required' => 'Nama produk wajib diisi',
                'min_length' => 'Nama produk minimal 3 karakter',
            ],
            'price' => [
                'required' => 'Harga wajib diisi',
                'numeric' => 'Harga harus berupa angka',
                'greater_than' => 'Harga harus lebih dari 0',
            ],
            'stock' => [
                'required' => 'Stok wajib diisi',
                'integer' => 'Stok harus berupa angka',
                'greater_than_equal_to' => 'Stok tidak boleh kurang dari 1',
            ],
        ];
    }
}

```

Routes.php

```

<?php

namespace Config;

use CodeIgniter\Config\Routing;

$route = Services::routes();

if (file_exists(SYSTEMPATH . 'Config/Routes.php')) {
    require SYSTEMPATH . 'Config/Routes.php';
}

// Auth Routes
$route->get('/', 'Auth::login');
$route->get('/login', 'Auth::login');
$route->post('/login', 'Auth::doLogin');
$route->get('/register', 'Auth::register');
$route->post('/register', 'Auth::doRegister');

```

```
$routes->get('/logout', 'Auth::logout');

// Dashboard & CRUD Routes
$routes->get('/dashboard', 'Product::dashboard');
$routes->get('/get-data', 'Product::getData');
$routes->post('/save', 'Product::save');
$routes->delete('/delete/(:num)', 'Product::delete');
$routes->get('/chart-data', 'Product::getChartData');

if (file_exists(APPPATH . 'Config/Routes.php')) {
    require APPPATH . 'Config/Routes.php';
}
```

Cara Penggunaan

1. Login

- Masukkan username/email dan password
- Klik tombol Login

2. Registrasi

- Klik link Register
- Isi form registrasi
- Klik tombol Register

3. Menambah Produk

- Klik tombol Add Product
- Isi form nama produk, harga, dan stok
- Klik Save Product

4. Mengedit Produk

- Klik tombol Edit pada baris produk
- Ubah data pada form yang muncul
- Klik Save Product

5. Menghapus Produk

- Klik tombol Delete pada baris produk
- Konfirmasi penghapusan melalui SweetAlert2
- Produk akan terhapus secara permanen

6. Export Data

- Klik tombol Excel
- File akan otomatis terdownload

7. Logout

- Klik tombol Logout di pojok kanan atas
- Konfirmasi logout

Fitur Realtime

Semua statistik pada dashboard akan berubah secara otomatis ketika:

- Menambah produk baru
- Mengedit data produk (harga atau stok)
- Menghapus produk

Tabel data juga akan otomatis refresh tanpa perlu reload halaman berkat penggunaan AJAX dan jQuery.

Troubleshooting

Error 500 Internal Server Error

- Pastikan file `.env` sudah dikonfigurasi dengan benar
- Cek permission folder writable: `chmod -R 755 writable/`

Error Database

- Pastikan MySQL server berjalan
- Cek kredensial database di file `.env`
- Pastikan database sudah dibuat

Error Class not found

- Jalankan `composer dump-autoload`

Error Routes tidak ditemukan

- Pastikan file `app/Config/Routes.php` sudah benar
 - Hapus cache: `php spark cache:clear`
-

Referensi

- [CodeIgniter 4 Documentation](#)
- [Bootstrap 5 Documentation](#)
- [jQuery Documentation](#)
- [DataTables](#)
- [Chart.js](#)
- [SweetAlert2](#)
- [Font Awesome](#)
- [MySQL](#)
- Telkom University - Program Studi S1 Informatika